

Izjave ExitLoop

Izlazi iz For i While petlji. Ima 3 oblika:

1. ExitLoop

Izlazi iz unutrašnje petlje koja je sadrži.

2. ExitLoop -

Izlazi iz unutrašnje petlje koja je sadrži i izlazi iz njene matične petlje.

Znak minus znači povratak za još jedan nivo. Možete koristiti -- da biste izašli iz dve matične petlje i možete dodati još znakova minusa po potrebi, ali praktično nećete imati više od tri ugnježdene petlje, a ako ste napisali četiri i želite da izađete iz svih njih, sledeći oblik će to učiniti.

3. ExitLoop *

Izlazi iz svih ugnježdenih petlji bez obzira na njihov broj. Na primer:

```
For I = 1 To 3
For J = 1 To 3
K = 1
While K <= 4
For N = 1 To 5
TextWindow.WriteLine(Text.Format("[1],[2],[3],[4]", {I, J, K, N}))
ExitLoop
Next
K = K + 1
Wend
Next
Next
```

Pokrenite ovaj kod i pogledajte rezultate. Pokušajte još 3 puta nakon što promenite komandu ExitLoop u ExitLoop-, ExitLoop-- i ExitLoop* respektivno, i uporedite rezultate.

```
(1,1,1,1)
(1,1,2,1)
(1,1,3,1)
(1,1,4,1)
(1,2,1,1)
(1,2,2,1)
(1,2,3,1)
(1,2,4,1)
(1,3,1,1)
(1,3,2,1)
(1,3,3,1)
(1,3,4,1)
(2,1,1,1)
(2,1,2,1)
(2,1,3,1)
(2,1,4,1)
(2,2,1,1)
(2,2,2,1)
(2,2,3,1)
(2,2,4,1)
(2,3,1,1)
(2,3,2,1)
(2,3,3,1)
(2,3,4,1)
(3,1,1,1)
(3,1,2,1)
(3,1,3,1)
(3,1,4,1)
(3,2,1,1)
(3,2,2,1)
(3,2,3,1)
(3,2,4,1)
(3,3,1,1)
(3,3,2,1)
(3,3,3,1)
(3,3,4,1)
```

Naredbe ContinueLoop

Izlazi iz trenutne iteracije i nastavlja sledeću od For i While petlji. Ima 3 oblika:

1. ContinueLoop

Nastavlja sledeću iteraciju unutrašnje petlje koja je sadrži.

2. ContinueLoop -

Izlazi iz unutrašnje petlje koja je sadrži i nastavlja sledeću iteraciju njene matične petlje. Znak minus znači povratak za još jedan nivo. Možete koristiti -- da biste nastavili dve matične petlje i možete dodati još znakova minusa po potrebi, ali praktično nećete imati više od tri ugnježdene petlje, a ako ste napisali četiri i želite da ih sve nastavite, sledeći oblik će to učiniti.

3. ContinueLoop *

Nastavlja sledeću iteraciju većine spoljašnjih petlji bez obzira na broj ugnježdenih petlji.

```
For I = 1 To 3
For J = 1 To 3
K = 1
While K <= 4
For N = 1 To 5
TextWindow.WriteLine(Text.Format("[1],[2],[3],[4]", {I, J, K, N}))
ContinueLoop
Next
```

```
K = K + 1
Wend
Next
Next
```

Pokrenite gornji kôd i pogledajte rezultate. Pokušajte još 3 puta nakon što promenite komandu ContinueLoop u ContinueLoop-, ContinueLoop-- i ContinueLoop* respektivno, i uporedite rezultate.

Imajte na umu da će korišćenje ContinueLoop- pretvoriti petlju u beskonačnu, jer preskače unutrašnju for petlju i skače na početak while petlje, bez izvršavanja naredbe $K = K + 1$ na njenom kraju, što će zauvek učiniti while uslov ($K \leq 4$) istinitim!

Zato budite oprezni i ne zaboravite da ažurirate while uslov petlje pre nego što je nastavite. Na primer, možete izbeći beskonačnu petlju koristeći ovo:

```
K = K + 1
ContinueLoop-
```

Naredbe za vraćanje – Return Statements

Opciono možete koristiti komandu Return u potprogramu da biste odmah izašli iz njega. Ona zapravo preskače na liniju EndSub, tako da se bilo koja linija koda koja sledi liniju Return nikada neće izvršiti.

Praktično, ova komanda se koristi unutar If naredbe da bi se kontrolisalo kada treba izaći iz potprograma. Na primer:

```
Sub Print(msg)
If msg = "" Then
Return
EndIf
TextWindow.WriteLine(msg)
EndSub
```

Komanda Return se koristi i u funkcijama, ali nije optimalna, jer je to jedini način da se vrati vrednost funkcije.

Ona takođe odmah izlazi iz funkcije, ali ovde stavlja njenu povratnu vrednost na stek, tako da pozivalac može da je pročita (ili ignoriše). Na primer:

```
TextWindow.WriteLine({Divide(4, 2), Divide(1, 0)})
Function Divide(a, b)
If b = 0 Then
Return "Ne može se deliti sa 0"
EndIf
Return a / b
EndFunction
```

2
Ne može se deliti sa 0

Kada pokrenete gornji kod, ispisaće se:

Izjave za poziv podprograma

Da biste pozvali podprogram, samo napišite njegovo ime, a zatim listu argumenata. Ako podprogram nema parametre, morate koristiti prazne zagrade () posle imena. Na primer:

```
SayHello()
Print("Zdravo svb!")
Sub SayHello()
TextWindow.WriteLine("Zdravo")
EndSub
Sub Print(msg)
TextWindow.WriteLine(msg)
```

EndSub

Imajte na umu da funkciju možete pozvati i kao da je podprogram, što znači ignorisanje njene povratne vrednosti. Na primer, ovaj kôd poziva funkciju ShowMsg dva puta: Jednom kao poziv metode da proveri njenu povratnu vrednost, a drugi put kao poziv potprograma ignorišući njenu povratnu vrednost:

```
If ShowMsg("Da li ste zadovoljni sa sVB?") = DialogResult.Yes Then
ShowMsg("Hvala")
EndIf
Function ShowMsg(msg)
Return Dialogs.MsgBox("Pažnja!", msg)
EndFunction
```



Takođe možete isto da uradite sa bilo kojom sVB metodom, da biste ignorisali njenu povratnu vrednost. Na primer:

```
Dialogs.MsgBox("Test", "Zdravo sVB!")
```

If naredbe

If naredba vam omogućava da proverite uslov i donesete odluku na osnovu toga da li je tačan ili netačan. If naredba ima mnogo oblika:

- If... EndIf

U svom najjednostavnijem obliku, If naredba ima samo jedan blok koda (granu):

```
If uslov Then
' Kôd koji želite da se izvrši
' kada je uslov tačan
EndIf
```

Gde uslov može biti:

1. bilo koja bulova promenljiva. Na primer:

```
A = True
If A Then
TextWindow.WriteLine("A je Istina.")
EndIf
```

2. Bilo koji poziv funkcije ili metode ili bilo koji izraz svojstva koji vraća bulovu vrednost:

```
X = "12"
If Text.IsNumeric(X) Then
TextWindow.WriteLine("x sadrži broj.")
EndIf
```

3. bilo koje logičko poređenje koje uključuje binarne operatore =, <>, >, >=, < ili <=. Na primer:

```
X = 7
Y = 2
If X = Y Then
TextWindow.WriteLine("x = y")
EndIf
If X < 10 Then
TextWindow.WriteLine("x < 10")
EndIf
```

Imajte na umu da je korišćenje bulove promenljive ili povratne vrednosti metode kao uslova zapravo prečica za njihovo poređenje sa True:

```
A = True
If A = True Then
TextWindow.WriteLine("A je Tačno.")
```

```
EndIf
```

Ovo znači da možete proveriti da li string nije numerički ovako:

```
X = "12$"  
If X.IsNumeric = False Then  
TextWindow.WriteLine("x nije broj.")  
EndIf
```

4. Složeni uslov koji sadrži više uslova kombinovanih zajedno sa jednim ili više binarnih operatora And ili Or, kao što je:

```
X = 7  
Y = 2  
If X = Y Or X > 5 Then  
TextWindow.WriteLine("U redu.")  
EndIf  
If X > 0 And Y > 0 And X + Y < 10 Then  
TextWindow.WriteLine("Takođe u redu.")  
EndIf
```

Imajte na umu da:

1. Kada je levi uslov operatora And netačan, sVB shvata da je rezultat operacije netačan, pa ne procenjuje desni uslov jer njegova vrednost neće biti važna.

2. Kada je levi uslov operatora Or tačan, sVB shvata da je rezultat operacije tačan, pa ne procenjuje desni uslov jer njegova vrednost neće biti važna.

3. Kada koristite And i Or zajedno, trebalo bi da zatvorite neke operacije u zagrade da biste sVB-u rekli kojim redosledom treba da izvrši ove operacije. Na primer, sledeći izraz:

```
(X > 0 Ili Y > 3) I Z < 7
```

je drugačiji od ovog:

```
X > 0 Ili (Y > 3 I Z < 7)
```

- Naredba If..Else

Naredba If može opciono da sadrži blok Else, gde navodite kôd koji želite da se izvrši kada je uslov netačan. Na primer:

```
X = -1  
If X > 0 Then  
TextWindow.WriteLine("x je pozitivan broj.")  
Else  
TextWindow.WriteLine("x je negativan broj.")  
EndIf
```

- Naredba If..ElseIf

Naredba If može opciono da sadrži jedan ili više blokova ElseIf, gde možete da proverite dva ili više povezanih uslova. Kada se utvrdi da je prvi od tih uslova tačan, sVB izvršava svoj blok koda i prelazi na liniju EndIf bez provere ostalih uslova. Na primer, ovaj kôd proverava ocene studenta da bi naveo svoju konačnu ocenu:

```
Poeni = 55  
If Poeni < 30 Then  
Ocena = "Previše slabo"  
If Poeni < 50 Then  
Ocena = "Nedovoljno"  
If Poeni < 65 Then  
Ocena = "Prihvaćeno"  
If Poeni < 75 Then  
Ocena = "Dobro"  
If Poeni < 85 Then  
Ocena = "Vrlo dobro"  
If Poeni >= 85 Then
```

```
Ocena = "Odlično"
EndIf
TextWindow.WriteLine(Ocena)
```

- Ugneždene If naredbe

Blokovi koda za If, ElseIf i Else mogu da sadrže jednu ili više naredbi bilo kog tipa, uključujući If naredbe i druge blokove koda poput for petlji. I ovo može da ide dublje do bilo kog broja nivoa kako vam je potrebno. Da biste ovo videli u akciji, otvorite projekat „Simple Calculator 3“ iz fascikle sa primerima i pogledajte potprogram za validaciju:

```
Sub Validate(numTextBox)
c = Event.LastTextInput
If Text.IsNumeric(c) = False Then
If c = "-" Then
If Text.StartsWith(
numTextBox.Text, "-") Then
Sound.PlayBellRing()
Event.Handled = True
Else
numTextBox.Select(1, 0)
EndIf
ElseIf c = "." Then
If Tekst.Contains(
numTextBox.Tekst, ".") Then
Sound.PlayBellRing()
Event.Handled = True
EndIf
Else
Sound.PlayBellRing()
Event.Handled = True
EndIf
EndIf
EndSub
```

For izjave

Petlja for se koristi za ponavljanje zadatka određeni broj puta. Koristi brojač (takođe nazvan iterator) koji počinje sa datom početnom vrednošću i povećava se za datu vrednost koraka u svakoj iteraciji petlje, i to nastavlja dok ne dostigne konačnu vrednost, koja će biti poslednja iteracija petlje. Ovo je opšti oblik petlje For:

```
For Counter = start To kraj Step priraštaj
' zadatak koji treba ponoviti
Next
```

- Next i EndFor

Radi unazadne kompatibilnosti sa SB, ključna reč EndFor se i dalje može koristiti umesto Next, ali vas ohrabrujem da koristite Next, jer daje značenje ponavljanja i kruženja kroz petlju, dok EndFor može biti zbunjujuće jer daje značenje završetka i izlaska, što može implicirati da se „petlja ovde završava“, a ne samo „kraj bloka for je ovde“!

- Brojač petlje (iterator)

Brojač može biti bilo koja promenljiva sa bilo kojim imenom koje izaberete.

Nakon što se petlje završe, vrednost brojača će biti veća od konačne vrednosti. Ovaj kôd, na primer, će ispisati 11:

```
For n = 1 To 10 Step 2
Next
TextWindow.WriteLine(n) ' 11
```

Promenljiva brojača je tipa Double, što vam omogućava da je koristite kao objekat i direktno pristupite metodama tipa Math iz nje:

```
For I = 1 To 2 Step 0.2
  TextWindow.WriteLine(I.Round)
Next
```

Ako je petlja for napisana na nivou datoteke (globalno domen), brojač će biti globalna promenljiva i može se promeniti iz bilo kog potprograma koju pozovete iz tela petlje, što će promeniti način na koji se petlja izvršava i može prouzrokovati da se petlja izvršava zauvek! Na primer, sledeća petlja će se kretati korak po koraku 2 iako smo je definisali da se kreće korak po koraku 1, jer smo menjali brojač u podprogramu Test:

```
For Counter = 1 To 10 Step 1
  Test()
Next
Sub Test()
  TextWindow.WriteLine(Counter)
  Counter = Counter + 1
EndSub
```

1
3
5
7
9

Ako je petlja definisana u podprogramu ili funkciji, njen brojač će uvek biti lokalna promenljiva i sakriće svaku globalnu promenljivu sa istim imenom. Na primer, sledeći kôd (koji možete isprobati u datoteci „loop iterator domain.sb“ u folderu sa primerima), sadrži dva brojača sa imenom i, ali jedan od njih je globalni dok je drugi lokalni, i svaki od njih je izolovan od drugog:

```
X = {"1. Prva stavka", "2. druga stavka", "3. Treća stavka"}
Za I = 1 Do 3      ' I je globalna promenljiva
  TextWindow.WriteLine(X[I])
  WriteSubItems()
Next
Sub WriteSubItems()
  y = {"a" Prva podstavka", "b" druga podstavka", "c" treća podstavka"}
  Za i = 1 Do 3    ' "i" je lokalna promenljiva i ona skriva globalnu promenljivu sa istim imenom "I"
    TextWindow.WriteLine(" " + y[i])
  Next
EndSub
```

- Korak petlje – Step

Klauzula Step for petlje je opcionalna, što znači da je možete izostaviti, i u ovom slučaju korak će podrazumevano biti 1:

```
For i = 1 To 5
  TextWindow.WriteLine(i)
Next
```

5
4
3
2
1

Korak može biti negativan broj, kako biste mogli da se vratite unazad od velikog broja do manjeg:

```
For i = 5 To 1 Step -1
  TextWindow.WriteLine(i)
Next
```

5
4
3
2
1

Ne koristite korak 0, u suprotnom će se petlja nastavljati beskonačno i nikada se neće završiti!

Ako je konačna vrednost manja od početne vrednosti, a korak je pozitivan, petlja će biti pre-skočena i zadatak se nikada neće izvršiti. Isto će se desiti ako je konačna vrednost veća od početne vrednosti a korak je negativan.

Ako su konačna vrednost i početna vrednost jednake, petlja će se ponoviti samo jednom.

- Ugneždene For petlje

Možete napisati dve ili više ugnežđenih petlji (petlji unutar petlji), gde svaka od njih mora

imati jedinstveno ime brojača, jer sve pripadaju istom opsegu promenljive (globalnom ili lokalnom):

```
For x = 1 To 2
For y = 1 To 2
For z = 1 To 2
  TextWindow.WriteLine(Text.Format("[1],[2],[3]", {x, y, z}))
Next
Next
Next
```

```
(1,1,1)
(1,1,2)
(1,2,1)
(1,2,2)
(2,1,1)
(2,1,2)
(2,2,1)
(2,2,2)
```

Izlaz iz petlje – ExitLoop

Možete koristiti naredbu `ExitLoop` unutar tela `For` petlje da biste odmah izašli iz petlje i prešli na liniju koda koja sledi sledeću liniju. Ovaj kôd, na primer, će ispisati 6, a ne 11, jer se iz petlje izlazi kada je `n` jednako 6, što čini uslov `if` tačnim:

```
For n = 1 To 10
If n > 5 Then
ExitLoop
EndIf
Next
TextWindow.WriteLine(n)
```

Ako želite da izađete iz matične petlje, koristite `ExitLoop-`, a ako želite da izađete iz roditelja matične petlje, koristite `ExitLoop--`, i tako dalje. Za primer, pogledajte datoteku „ExitLoop.sb“ u folderu sa primerima.

Ako želite da izađete iz najspoljašnje petlje da biste odjednom izašli iz svih ugnežđenih petlji, koristite `ExitLoop*`. Na primer, ovaj kôd neće ispisati ništa na ekranu, jer komanda `ExitLoop*` izlazi iz svih petlji pre izvršavanja metode `WriteLine`, ali ako ste uklonili (ili komentarisali) liniju `ExitLoop*`, videćete 1 i 2 ispisane u dve linije:

```
For x = 1 To 2
For y = 1 To 2
For z = 1 To 2
ExitLoop *
Next
Next
Next
TextWindow.WriteLine(x)
Next
```

Nastavi petlju – Continue Loop

Možete koristiti naredbu `ContinueLoop` unutar tela `For` petlje da biste odmah prešli na sledeći red kako biste povećali brojač i nastavili sledeću iteraciju. Na primer, sledeći kôd će ispisati sve brojeve od 1 do 10 osim 7 jer ispunjava uslov koji izvršava komandu `ContinueLoop`:

```
For n = 1 To 10
If n.Remainder(2) = 1 And
n Mod 5 = 2 Then
ContinueLoop
EndIf
TextWindow.WriteLine(n)
Next
```

```
1
2
3
4
5
6
8
9
10
```

Ako želite da nastavite roditeljsku petlju, koristite `ContinueLoop-`, a ako želite da izađete iz roditelja roditeljske petlje, koristite `ContinueLoop--`, i tako dalje. Za primer, pogledajte datoteku "ExitLoop.sb" u folderu sa primerima.

Ako želite da izađete iz najspoljašnje petlje da biste odjednom izašli iz svih ugnežđenih petlji, koristite ContinueLoop*.

Stabilnost petlje

U SB, konačne vrednosti i vrednosti Step for petlje mogu se menjati u telu petlje. Na primer, ovo je teoretski beskonačna petlja u SB:

```
n = 5
For i = 1 To n
  TextWindow.WriteLine(i)
  n = n + 1
EndFor
```

1
2
3
4
5

Ali u sVB, konačne vrednosti i vrednosti koraka se postavljaju jednom i njihova promena u telu petlje neće imati uticaja na petlju. To znači da će gornja for petlja ispisati brojeve od 1 do 5 i završiti se normalno, a promena „n = n + 1“ u telu petlje je nebitna. Ovo čini sVB konzistentnim sa VB6 i VB.NET, a takođe je i dobra optimizacija, kako bi se izbeglo ponovno izračunavanje izraza final i step u svakoj iteraciji petlje.

Menjanje brojača (ovde n) u telu petlje je veoma opasno jer može oštetiti petlju ili je pretvoriti u petlju-u-petlji, ali ponekad može biti korisno, kao kada treba da obradite sledeći element niza sa trenutnim elementom, preskočite sledeći element ručnim povećanjem brojača za 1.

Zato budite oprezni kada manipulišete brojačem u telu petlje. sVB kompajleri primenjuju metod Text.ToNumber na početni, završni i inkrementalni izraz for petlji kako bi se osiguralo da su to validni brojevi. Ovaj metod će vratiti 0 ako je bilo koji od ovih izraza string, ali ako koristite jedan znak, ovaj metod će vratiti njegovu Unicode vrednost. Na primer, sledeća petlja će prikazati brojeve od 97 do 122, što su Unicode vrednosti slova od a do z:

```
For i = "a" To "z"
  TextWindow.WriteLine(i)
Next
```

Ovo je korisno kada želite da petljom unesete niz slova. Na primer, ova petlja će prikazati slova od F do M:

```
For i = "F" To "M"
  TextWindow.WriteLine(i.ToChar())
Next
```

F
G
H
I
J
K
L
M

ForEach naredbe

ForEach naredba je poseban oblik For naredbe. Koristi se za iteraciju kroz stavke nizova u obliku:

```
ForEach stavka In nizu
  ' nešto da se uradi
Next
```

Identifikator stavke može imati bilo koje ime koje želite i sadržaće prvu stavku niza na početku, zatim će sadržati sledeću stavku u sledećoj iteraciji i tako dalje. Na primer:

```
brojevi = {10, 20, 30, 40}
ForEach broj In brojevi
  TextWindow.WriteLine(broj)
Next
```

10
20
30
40

ForEach naredba je lakša za rad sa nizovima sa string ključevima, kada samo želite da dobijete

vrednosti stavki bez obzira na njihove ključeve. Na primer:

```
Niz[1] = "Jedan"
Niz ["msg"] = "Zdravo"
Niz!Name = "Ahmad"
ForEach stavka In Niz
TextWindow.WriteLine(stavka)
Next
```

```
Jedan
Zdravo
Ahmad
```

Imajte na umu da sVB ne može da zaključi tip iteratora (kao što su broj i stavka u gornjim primerima), jer niz može da sadrži stavke različitih tipova. Ali možete reći sVB-u šta da očekuje primenom pravila konvencija imenovanja na identifikator iteratora. Na primer, korišćenje strName (ili strIme) kao iteratora govori sVB-u da je to string, tako da možemo koristiti Text metode preko njega:

```
imena = {"Adam", "Džon", "Jaser"}
ForEach strIme In imena
TextWindow.WriteLine(strIme.UpperCase)
Next

Imena = {"Adam", "Džon", "Jaser"}
ForEach StrIme In Imena
    TextWindow.WriteLine(StrIme)
Next
```

```
ADAM
DŽON
JASER
```

```
Adam
Džon
Jaser
```

Takođe možete koristiti ForEach za iteraciju kroz sve znakove stringa:

```
ForEach C In "Zdravo"
TW.WriteLine(C)
Next
```

```
Z
d
r
a
v
o
```

Dalje, možete koristiti ForEach za iteraciju kroz sve cifre broja: !!!

```
ForEach D In 100 + 25    'ovo ne radi!!!
TW.WriteLine(D)
Next

ForEach C In "125"      'ovo radi!!!
TW.WriteLine(C)
Next
```

```
1
2
5
```

Imajte na umu da se sva ostala pravila For naredbe primenjuju na ForEach naredbu, kao što je korišćenje ExitLoop i ContinueLoop naredbi u telu ForEach.

While naredbe

While petlja se koristi za ponavljanje zadatka sve dok je određen uslov dostupan (True). Ovo je opšti oblik while petlje:

```
While uslov
    ' zadatak koji treba ponoviti
Wend
```

- Wend i EndWhile

Radi kompatibilnosti sa prethodnim verzijama, ključna reč EndWhile se i dalje može koristiti umesto Wend, ali vas ohrabrujem da koristite Wend radi doslednosti sa Visual Basic-om. Tačno je da Visual Basic .NET koristi EndWhile, ali mi se ne sviđa jer deluje zbunjujuće jer daje značenje završetka i izlaska, što može implicirati da se „petlja ovde završava“, a ne samo „kraj while bloka je ovde“!

Wend je ovde najbolji izbor, jer daje značenje ponavljanja i kruženja kroz petlju.

- *Beskonačna While petlja*

Lako je dobiti beskonačnu vrednost koristeći While, jer je sve što vam je potrebno da obezbedite uslov koji je uvek tačan, kao što je $1 = 1$, ili možete jednostavno koristiti True:

```
While True
  TextWindow.WriteLine("Beskonačna petlja!")
Wend
```

Izazov ovde nije imati beskonačnu while petlju, jer morate osigurati da while uslov može postati netačan u nekom trenutku. Na primer, ovo je beskonačna petlja, jer ne sadrži instrukcije za ažuriranje vrednosti n, pa će uslov ostati tačan zauvek:

```
n = 1
While n <= 5
  TextWindow.WriteLine(n)
Wend
```

Izgleda da je gornji kôd trebalo da ispiše brojeve od 1 do 5 na ekranu, ali će nastaviti da ispisuje 1 beskonačno!

Dakle, da bismo to popravili, moramo povećati n u telu petlje, tako da uslov postane netačan kada n dostigne 6 i dobijemo 5 različitih brojeva na ekranu:

```
n = 1
While n <= 5
  TextWindow.WriteLine(n)
  n = n + 1
Wend
```

- *Ugneždene petlje*

Možete imati dve ili tri ili više ugnežđenih while petlji (petlji unutar petlji). Na primer:

```
x = 1
While x < 3
  y = 1
  While y < 3
    z = 1
    While z < 3
      TextWindow.WriteLine(Text.Format("[1],[2],[3]", {x, y, z}))
      z = z + 1
    Wend
    y = y + 1
  Wend
  x = x + 1
Wend
```

```
(1,1,1)
(1,1,2)
(1,2,1)
(1,2,2)
(2,1,1)
(2,1,2)
(2,2,1)
(2,2,2)
```

Takođe je u redu pisati ugneždene petlje for i while po potrebi.

- *Izlaz iz petlje – ExitLoop*

Možete koristiti naredbu ExitLoop unutar tela petlje while da biste odmah izašli iz petlje i prešli na liniju koda koja sledi liniju Wend.

Sledeći program će stalno tražiti od korisnika da unese broj za ispis brojeva od 1 do njega (uključujući i taj broj), a petlja će se završiti samo kada korisnik unese 0 ili prazan red:

```
While True
  TextWindow.Write("Unesite broj")
  TextWindow.Write(" (ili 0 exit):")
  N = TextWindow.ReadNumber()
  If N = 0 Then
    ExitLoop
  EndIf
  S = 0
  For I = 1 To N
    S = S + 1
```

```
Unesite broj (ili 0 exit):12
1 2 3 4 5 6 7 8 9 10 11 12
Unesite broj (ili 0 exit):
```

```

TextWindow.Write(S + " ")
Next
TextWindow.WriteLine("")
Wend

```

Ako želite da izađete iz roditeljske petlje, koristite `ExitLoop-`, a ako želite da izađete iz roditelja roditeljske petlje, koristite `ExitLoop--`, i tako dalje.

Ako želite da izađete iz najspoljašnje petlje (da biste izašli iz svih ugnežđenih petlji odjednom) koristite `ExitLoop*`.

- Nastavak petlje – Continue Loop

Možete koristiti naredbu `ContinueLoop` unutar tela `while` petlje da biste odmah prešli na `Wend` liniju i nastavili sledeću iteraciju. Ali to može biti opasno ako dovodi do preskakanja linije koja ažurira uslov `while` petlje (kao što je povećanje vrednosti promenljive ili tako nešto). Zato obavezno ažurirajte uslov ako je potrebno pre nego što nastavite petlju.

Na primer, sledeći kôd će ispisati sve brojeve od 1 do 10 osim 7 jer ako ispunjava uslov koji izvršava komandu `ContinueLoop`:

```

N = 1
While N < 11
If N Mod 2 = 1 And
N.Remainder(5) = 2 Then
N = N + 1
ContinueLoop
EndIf
TextWindow.WriteLine(N)
N = N + 1
Wend
TextWindow.WriteLine("Kraj.")

```

Imajte na umu da ako uklonite red `N = N + 1` iz tela `if` naredbe, petlja se nikada neće završiti i videćete brojeve od 1 do 6 na ekranu, ali naredba "Kraj." se neće pojaviti jer petlja i dalje radi zauvek!

Ako želite da nastavite roditeljsku petlju, koristite `ContinueLoop-`, a ako želite da izađete iz roditelja roditeljske petlje, koristite `ContinueLoop--`, i tako dalje.

Ako želite da izađete iz najspoljašnje petlje da biste odjednom izašli iz svih ugnežđenih petlji, koristite `ContinueLoop*`.

Izjave za podprograme

Podprogram je imenovani blok koda, tako da ga možete izvršiti onoliko puta koliko vam je potrebno samo pozivanjem njegovog imena. Na primer:

```

PrintTime()
Program.Delay(1000)
PrintTime()
Sub PrintTime
TextWindow.WriteLine(Date.Now)
EndSub

```

Podrška za uređivač

Gornja desna padajuća lista u uređivaču koda prikazuje imena svih podprograma i funkcija u trenutnoj datoteci, što vam olakšava navigaciju do bilo koje od njih jednostavnim klikom na njeno ime.

Lista takođe sadrži komandu Add New Sub – *Dodaj novi podprogram*, na koju možete kliknuti da biste dodali nov prazan podprogram u datoteku, koja izgleda ovako:

```
'-----  
Sub NewSub_1()  
EndSub
```

NewSub_1 će biti izabran i možete direktno uneti ime podprograma. Možete definisati listu parametara za podprogram, ovako:

```
Print("Udaljenost", 120)  
Sub Print(ime, vrednost)  
TextWindow.WriteLine("Naziv=" + ime + ", Vrednost=" + vrednost)  
EndSub
```

Možete koristiti pravila konvencija imenovanja da biste omogućili sVB-u da zaključi tipove parametara. Na primer:

```
Sub Print(strIme, dvostrukaVrednost)  
TextWindow.WriteLine("Naziv=" + strIme.MalaSlova + ", Vrednost=" + dvostrukaVrednost.Abs)  
EndSub
```

Imajte na umu da možete koristiti naredbu Return unutar tela podprograma da biste odmah izašli iz podprograma.

- Rukovaoci događaja - Event Handlers

Podprogram može da radi kao rukovalac događajima, što znači da će se ovaj podprogram pozivati svaki put kada se taj događaj pokrene.

Rukovalac događaja možete dodati u uređivač koda na bilo koji od ovih načina:

1. Dvaput kliknite na obrazac ili bilo koju kontrolu u dizajneru obrazaca. Ovo će preći na uređivač koda i dodati rukovalac za podrazumevani događaj kontrole, kao što je Button1_OnClick.
2. Izbor imena kontrole iz gornje leve padajuće liste. Ovo će prikazati događaje kontrole u desnoj padajućoj listi i možete kliknuti na bilo koji od njih da biste dodali njegov rukovalac u datoteku.

3. Korišćenje konvencije imenovanja za ručno kreiranje potprograma. sVB koristi istu VB6 konvenciju imenovanja za rukovaoce događajima, a to je: ControlName_EventName.

4. Postavljanje obrađivača događaja pomoću koda. Na primer, ovaj kôd će postaviti potprogram Click da bude obrađivač događaja form.OnClick:

```
Me.Text = Text.Format(  
Me.OnClick = Click  
Sub Click()  
"([1],[2])", {Me.MouseX, Me.MouseY})  
EndSub
```

- Zašto su nam potrebni potprogrami?

Korišćenje potprograma ima sledeće prednosti:

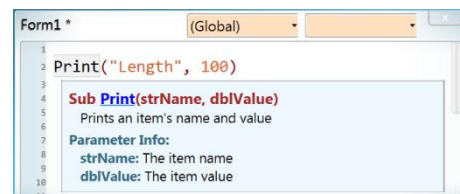
1. Čitljivost. Korišćenje potprograma vam omogućava da podelite program na imenovane jedinice, što ga čini lakšim za razumevanje.
2. Sažetost. Pozivanje imena potprograma više puta je manje opširno nego ponavljanje istog koda više puta na različitim mestima.
3. Kontrolabilnost. Možete kontrolisati kako potprogram radi putem svojih ulaznih parametara, tako da isti blok koda može da obavlja malo drugačiji posao u skladu sa argumentima koje ste mu poslali.
4. Održavanje. Možete lako popraviti i izmeniti kôd na samo jednom mestu (u potprogramu) umesto da se mučite po celom kodu.

- Dokumentacija potprograma

Možete koristiti dokumentaciju sa ugrađenim komentarima da biste pružili informacije o potprogramu i njegovim parametrima.

Definiciju potprograma možete dati u redovima iznad nje i na kraju njenog zaglavlja. Takođe možete da prelomite parametre preko više redova i date definicije za neke ili sve njih. Na primer:

```
'Ispisuje ime i vrednost stavke'  
Sub Print(  
  strName, ' Ime stavke  
  dblValue ' Vrednost stavke  
)  
  TextWindow.WriteLine(  
    "Name=" + strName.LowerCase +  
    ", Value=" + dblValue.Abs)  
EndSub
```



Izjave za funkcije

Funkcija je potprogram koji može da vrati rezultat. Može imati nula ili više parametara i koristi izjavu Return da bi vratila vrednost. Na primer:

```
X = Sum(1, 2)  
  
Funkcija Sum(x, y)  
  Return x + y  
EndFunction
```

Podrška za uređivač

Gornja desna padajuća lista u uređivaču koda prikazuje imena svih potprograma i funkcija u trenutnoj datoteci, što vam olakšava navigaciju do bilo koje od njih jednostavnim klikom na njeno ime.

Lista takođe sadrži komandu Add New Function – *Dodaj novu funkciju*, na koju možete kliknuti da biste dodali novu praznu funkciju u datoteku, koja izgleda ovako:

```
'-----  
Funkcija NewFunc_1()  
  Return 0  
EndFunction
```

NewFunc_1 će biti izabrana i možete direktno uneti ime funkcije.

- Funkcije za testiranje

Funkcija za testiranje je normalna funkcija, ali njeno ime počinje prefiksom Test_ i vraća string koji vam govori o rezultatu testiranja dela vašeg obrasca.

Za više informacija o tome kako pisati i pokretati funkcije za testiranje, pogledajte objašnjenje i primere koji su dati uz metod Form.RunTests.

- Dokumentacija funkcija

Možete koristiti dokumentaciju ugrađenih komentara da biste dali definicije za funkciju i njene parametre, baš kao što to radite sa potprogramima.

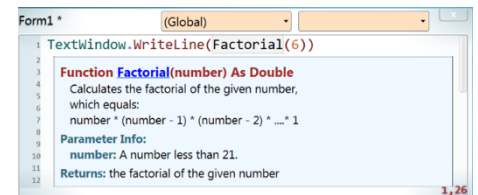
Takođe možete dati definiciju za povratnu vrednost funkcije nakon njene desne zagrade. Na primer:

```
' Izračunava faktorijel datog broja,  
' što je jednako:
```

```

' broj * (broj - 1) * (broj - 2) * .... * 1
Funkcija Factorial(
broj ' Broj manji od 21.
) ' faktorijel datog broja
If broj < 2 Then
Return 1
EndIf
Return broj * Factorial(broj - 1)
EndFunkcion

```



Imajte na umu da je funkcija `Factorial` rekurzivna funkcija, jer rekurzivno poziva samu sebe dok određeni uslov to ne zaustavi. Budite oprezni kada kreirate rekurzivnu funkciju da ne pozivate samu sebe beskonačno jer će to srušiti vašu aplikaciju sa greškom „Out of stack memory“.

- Zaključivanje tipa povratka funkcije

Dobra stvar koju možete primetiti sa gornje slike je da je sVB uspeo da zaključi tip povratne vrednosti funkcije kao `Double`. sVB traži sve povratne naredbe korišćene u funkciji i pokušava da zaključi njihove tipove, i ako svi pripadaju istom tipu, koristi ga kao povratni tip funkcije. U suprotnom koristi širi tip koji može da uključi sve ostale tipove (kao što je tip `Control` koji uključuje `TextBox`, `Button` i sve ostale kontrole).

Ako takav tip ne postoji, sVB ne uspeva da zaključi povratni tip funkcije, što se inače očekuje u rekurzivnoj funkciji jer njen povratni tip zavisi od sopstvenog povratnog tipa, ali srećom, operator množenja važi samo za članove i zato je sVB uspeo da zaključi tip. Dakle, generalno, ako sVB nije uspeo da zaključi povratni tip, trebalo bi da mu pomognete, koristeći pravila konvencija imenovanja da biste definisali jednu ili više promenljivih tako da sVB zna njihove tipove, a zatim ih koristite u povratnim naredbama funkcije!